Efficient Mining of Low-Utility Sequential Patterns

Comparison of Naive and Optimized Algorithms

Aditya Dhamala | Registration No. 22BDS0424

Problem & Core Concepts

Sequential Pattern Mining

Discovers frequent subsequences in ordered transaction data – allowing **non-contiguous** item skipping.

Utility & Low-Utility Sequences

Each item carries a **utility value** (profit, cost, or weight). A **low-utility sequence** has aggregate utility ≤ **max_utility threshold**.

Threshold: max_utility = **22** (SIGN dataset)

Algorithm Comparison

LUSPMb — Naive

- Generates **all** possible subsequences
- No pruning – exhaustive enumeration
- High computational and memory cost

LUSPMe — Optimized

- Applies **utility-based pruning** early
- Eliminates unpromising candidates
- Faster and memory-efficient

Metric	LUSPMb (Naive)	LUSPMe (Optimized)
Candidates Generated	1,048,576	782
Runtime	1.886 s	0.038 s
Memory Usage	~26 MB	~9 MB
Patterns Found	411	411

Results & Conclusion

49x

Faster Runtime

1.886 s → 0.038 s

1339x

Fewer Candidates

1,048,576 → 782

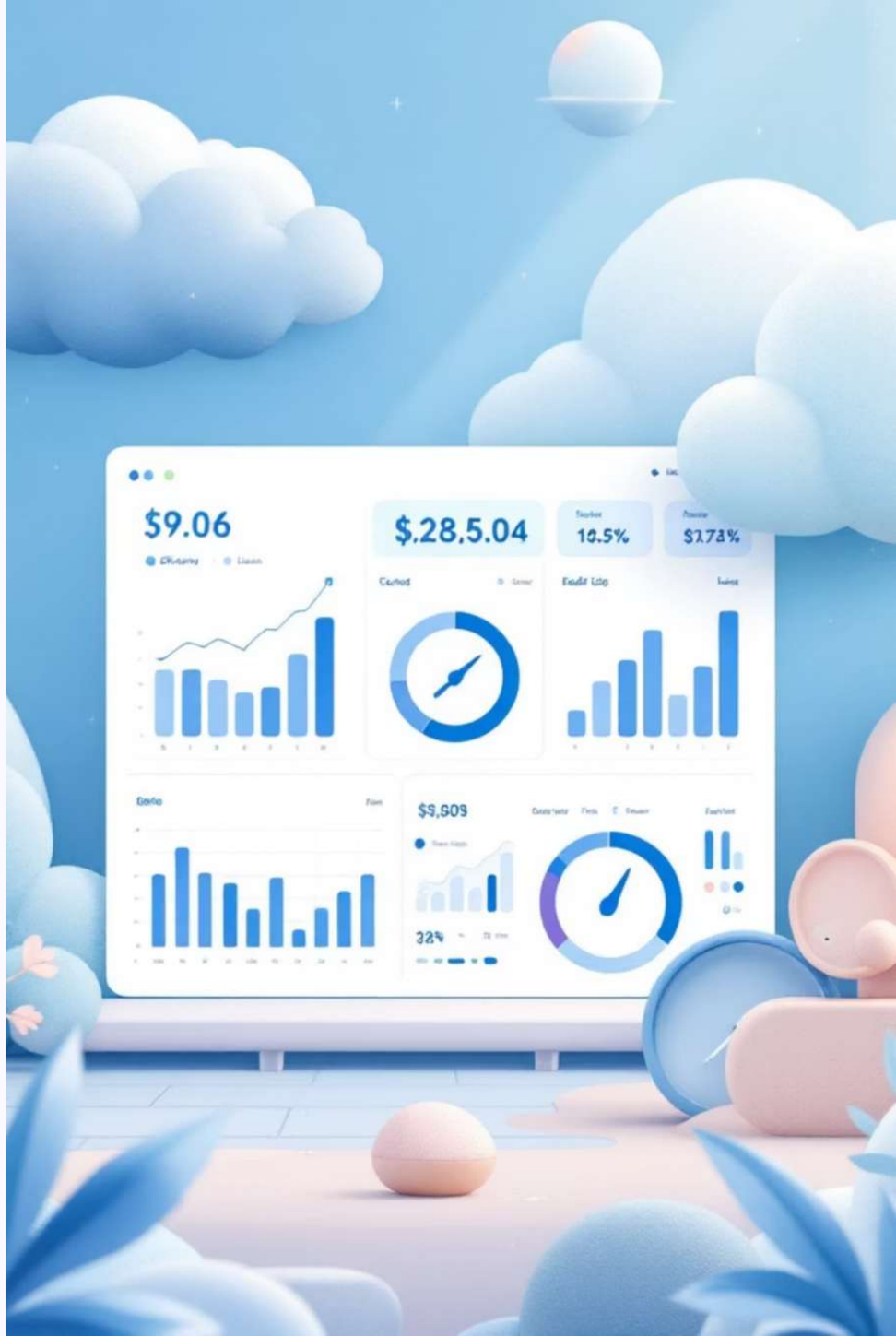
3x

Less Memory

~26 MB → ~9 MB

411

Identical Patterns

Both algorithms agree

Key Takeaway: LUSPMe produces **identical results** to LUSPMb while drastically cutting candidates, runtime, and memory – confirming that **pruning is essential** for scalable low-utility sequential pattern mining.